Sesión 7

Gradient Boosting: XGBoost, LightGBM y el estado del arte en tablas

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

De bagging a boosting

- **Random Forest (bagging)**: árboles en paralelo, promedian predicciones $\rightarrow$ reducen varianza.
- **Boosting**: árboles en secuencia, cada uno corrige errores del anterior $\rightarrow$ reduce sesgo.
- La corrección iterativa permite alcanzar mayor precisión, pero con mayor riesgo de sobreajuste.

AdaBoost: el precursor

Idea principal

1. Asignar pesos iguales a todas las observaciones.
2. Entrenar un clasificador débil (ej. árbol de profundidad 1, "stump").
3. Aumentar el peso de las observaciones mal clasificadas.
4. Repetir y combinar por votación ponderada.

- **Limitaciones**: sensible a ruido y outliers; no se generaliza fácilmente a otras funciones de pérdida.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Marco general

Objetivo: minimizar $\mathcal{L}(F) = \sum_{i=1}^{n} L(y_i, F(x_i))$ con L diferenciable.

Construcción secuencial

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

- F_{m-1}: modelo hasta la iteración anterior.
- h_m: árbol añadido en la iteración m.
- η: **tasa de aprendizaje (learning rate)**.

Ajuste de residuos (pérdida cuadrática)

Para $L(y, F) = \frac{1}{2}(y - F)^2$, el gradiente negativo es:

$$r_{im} = -\left.\frac{\partial L}{\partial F}\right|_{F=F_{m-1}(x_i)} = y_i - F_{m-1}(x_i)$$

- Cada nuevo árbol h_m se entrena para predecir los **residuos** del modelo actual.
- Luego se suma escalado por η.

Pérdida general: pseudo-residuos

Para una pérdida arbitraria, se usan los **gradientes negativos**:

$$g_{im} = -\left.\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}}$$

1. Calcular g_{im} para cada observación.
2. Ajustar un árbol h_m a los pares (x_i, g_{im}).
3. Actualizar $F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$.

Esto equivale a un paso de gradiente descendente en el espacio de funciones.

Algoritmo genérico de Gradient Boosting

1. Inicializar $F_0(x) = \arg\min_\gamma \sum_i L(y_i, \gamma)$ (una constante).
2. Para $m = 1$ hasta M:
   1. Calcular pseudo-residuos g_{im}.
   2. Ajustar árbol h_m a (x_i, g_{im}).
   3. Determinar paso ρ_m (o usar η fijo).
   4. Actualizar $F_m = F_{m-1} + \eta\, h_m$.
3. Salida $F_M(x)$.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

XGBoost (eXtreme Gradient Boosting)

- **Regularización**: añade penalización a la pérdida:

$$\mathcal{L} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2 + \alpha |w|$$

  (T: número de hojas, w: pesos de hojas)
- **Manejo nativo de valores faltantes**: aprende dirección óptima.
- **Early stopping** para evitar sobreajuste.
- **Paralelización** en la búsqueda de splits.

LightGBM (Light Gradient Boosting Machine)

- **GOSS** (Gradient-based One-Side Sampling): retiene observaciones con gradientes grandes y muestrea las de gradiente pequeño.
- **EFB** (Exclusive Feature Bundling): agrupa características mutuamente excluyentes.
- **Crecimiento leaf-wise** (por hoja) en lugar de level-wise: más preciso pero puede sobreajustar.
- **Soporte nativo para categóricas**.

CatBoost (Categorical Boosting)

- Especializado en variables categóricas.
- Target encoding con ordenamiento aleatorio para evitar leakage.
- Árboles simétricos que aceleran la inferencia.
- Ideal cuando hay muchas categóricas.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Random Forest vs Gradient Boosting

Característica	**Random Forest**	**Gradient Boosting**
Construcción	Paralela (bagging)	Secuencial (boosting)
Objetivo	Reducir varianza	Reducir sesgo
Sobreajuste	Robusto por naturaleza	Requiere regularización
Velocidad	Rápido (paralelo)	Puede ser más lento
Número de árboles	Menos necesario	Puede necesitar miles
Datos ruidosos	Robusto	Sensible
Rendimiento típico	Bueno	Excelente (con ajuste)

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Hiperparámetros comunes en XGBoost y LightGBM

XGBoost

- learning_rate (η)
- max_depth
- subsample
- colsample_bytree
- lambda (L2)
- alpha (L1)
- gamma

LightGBM

- learning_rate
- num_leaves (relacionado con profundidad)
- feature_fraction
- bagging_fraction
- lambda_l1, lambda_l2
- min_data_in_leaf

Ajuste de hiperparámetros

- Relación η y número de árboles: η pequeño requiere más árboles, pero mejora generalización.
- **Early stopping** sobre conjunto de validación para elegir el número óptimo de árboles.
- Búsqueda con `RandomizedSearchCV` o `GridSearchCV`.
- Para LightGBM, controlar `num_leaves` para evitar sobreajuste.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Clasificación: log-loss y AUC

Log-loss (entropía cruzada)

$$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)\right]$$

Penaliza predicciones seguras pero erróneas. Ideal para probabilidades bien calibradas.

AUC-ROC

Mide la capacidad de ranking, independiente del umbral. Útil en problemas desbalanceados.

Regresión: RMSE, MAE, RMSLE

RMSE

$$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$$

Sensible a outliers.

MAE

$$\frac{1}{n}\sum|y_i - \hat{y}_i|$$

Robusto a outliers.

RMSLE

$$\sqrt{\frac{1}{n}\sum(\log(y_i + 1) - \log(\hat{y}_i + 1))^2}$$

Penaliza más la infraestimación.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Predicción de clics (CTR) en publicidad online

- Datos masivos, alta cardinalidad, necesidad de velocidad.
- **LightGBM** es ideal por su eficiencia y manejo de categóricas.
- Métrica principal: log-loss o AUC.

Competencias Kaggle

- La mayoría de soluciones ganadoras en datos tabulares usan XGBoost, LightGBM o ensembles.
- Ejemplos: House Prices, Otto Group, Porto Seguro.

Modelado de riesgo crediticio

- Se necesita interpretabilidad y buen rendimiento.
- XGBoost proporciona importancia de características (gain).
- Se pueden usar pesos para clases desbalanceadas.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Contexto y datos

- Empresa de telecomunicaciones, 100,000 clientes, 50 variables.
- 15 % de abandono (clase positiva).
- Variables: demográficas, uso, reclamaciones, facturación.

Preprocesamiento

- Nulos: XGBoost los maneja internamente (no imputar).
- Categóricas: en LightGBM usar `categorical_feature`.
- División entrenamiento/validación/prueba estratificada.

Modelado y ajuste

- Probar XGBoost y LightGBM con búsqueda aleatoria de hiperparámetros.
- Usar **early stopping** con métrica AUC en validación.
- Resultados típicos: LightGBM suele ser ligeramente superior.

Modelo	AUC	Log-loss	F1 (clase 1)
XGBoost	0.832	0.352	0.612
LightGBM	0.845	0.341	0.631
Random Forest	0.801	0.382	0.582

Importancia de características

- Variables más importantes (gain):
  1. Número de reclamaciones últimos 6 meses.
  2. Factura media.
  3. Antigüedad del cliente.
  4. Consumo de datos último mes.
- **Acciones de negocio**: reducir reclamaciones, ofertas personalizadas, programas de fidelización.

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

XGBoost

```python
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)

dtrain = xgb.DMatrix(X_train, label=y_train)
dval = xgb.DMatrix(X_val, label=y_val)

params = {
    'objective': 'binary:logistic',
    'eval_metric': 'logloss',
    'max_depth': 5,
    'learning_rate': 0.05,
    'subsample': 0.8,
    'colsample_bytree': 0.8,
    'lambda': 1.0,
    'alpha': 0.1
}

model = xgb.train(params, dtrain, num_boost_round=1000,
                  evals=[(dval, 'validation')],
                  early_stopping_rounds=50, verbose_eval=50)

y_pred = model.predict(dval)
auc = roc_auc_score(y_val, y_pred)
```

LightGBM

```
import lightgbm as lgb
from sklearn.metrics import roc_auc_score

train_data = lgb.Dataset(X_train, label=y_train)
val_data = lgb.Dataset(X_val, label=y_val, reference=train_data)

params = {
    'objective': 'binary',
    'metric': 'auc',
    'boosting_type': 'gbdt',
    'num_leaves': 31,
    'learning_rate': 0.05,
    'feature_fraction': 0.8,
    'bagging_fraction': 0.8,
    'bagging_freq': 5,
    'lambda_l1': 0.1,
    'lambda_l2': 1.0
}

model = lgb.train(params, train_data, valid_sets=[val_data],
                  num_boost_round=1000,
                  callbacks=[lgb.early_stopping(50), lgb.log_evaluation(50)])

y_pred = model.predict(X_val)
auc = roc_auc_score(y_val, y_pred)
```

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Contexto del problema

- Plataforma de e-commerce (ej. Mercado Libre).
- Predecir si un usuario comprará después de hacer clic (conversión).
- Tasa de conversión baja (2 %).
- Datos masivos, muchas variables categóricas (usuario, producto, contexto).

Feature engineering específico

1. **Usuario**: frecuencia de compras, tasa de conversión histórica, promedio de gasto, días desde última compra.
2. **Producto**: precio relativo a categoría, popularidad (clics/ventas), antigüedad.
3. **Contexto**: hora (codificación cíclica), día de semana, dispositivo, ubicación.
4. **Interacciones**: usuario $\times$ categoría, precio $\times$ dispositivo, tiempo en página $\times$ precio.

Manejo del desbalance y evaluación

- Usar `scale_pos_weight` en XGBoost o `class_weight` en LightGBM.
- Métricas: AUC, log-loss, precisión/recall/F1 (ajustando umbral).
- Curva Precision-Recall especialmente informativa.
- Early stopping con conjunto de validación temporal.

Interpretabilidad y monitoreo

- Importancia de características (gain) para entender factores globales.
- SHAP para explicar predicciones individuales a equipos de negocio.
- Monitoreo en producción: asociar métricas técnicas a negocio (recall $\leftrightarrow$ ventas perdidas, precisión $\leftrightarrow$ clics desperdiciados).

Contenido

1. Introducción y Concepto de Boosting
2. Gradient Boosting Matemático
3. XGBoost, LightGBM, CatBoost
4. Comparación con Random Forest
5. Hiperparámetros Clave
6. Métricas de Evaluación
7. Aplicaciones Prácticas
8. Caso Integrado: Predicción de Churn
9. Implementación en Python
10. Caso Avanzado: Predicción de Compra (E-commerce)
11. Conclusión

Lo que hemos aprendido

Conceptos clave

- Gradient Boosting: construcción secuencial, pseudo-residuos, tasa de aprendizaje.
- XGBoost: regularización, manejo de nulos, early stopping.
- LightGBM: GOSS, EFB, crecimiento leaf-wise, soporte categóricas.
- Comparación con Random Forest.
- Hiperparámetros y ajuste.
- Métricas: log-loss, AUC, RMSE, MAE, RMSLE.
- Casos prácticos: CTR, churn, e-commerce.

¡Gracias!